Invention Disclosure: Scoped Telepathy for Multi-Agent Shared Memory Systems

1. Title

Scoped Event Routing with Self-Exclusion for Multi-Agent Shared Memory (Telepathy)

---

2. Inventors

- Paul Finton (AJ)

3. Date of Invention

April 28, 2026

---

4. Field of the Invention

This invention relates to real-time event distribution in multi-agent artificial intelligence systems. More specifically, it relates to a method for sharing memory state changes between autonomous AI agents operating on a shared memory store, where each agent receives only the changes relevant to its operational scope and never receives notifications of its own writes.

---

5. Background and Problem Statement

5.1 The Multi-Agent Problem

Modern AI agent systems (e.g., coding assistants, monitoring agents, personal assistants, DevOps agents) increasingly operate in teams. Multiple agents may be running simultaneously — a development agent writing code, a monitoring agent watching infrastructure, a personal agent managing user preferences — all working on behalf of the same user or organization.

These agents need shared context. When the development agent learns that a server has moved to a new IP address, the monitoring agent should know immediately, not wait for the user to repeat the information. When the personal agent learns the user prefers dark mode, the development agent should reflect that preference in its output.

5.2 Prior Art Approaches

Approach 1: Frozen context. Each agent loads its context once at startup and never updates it. This is simple but guarantees stale data. In a long-running session, one agent's context diverges from reality as other agents write new memories.

Approach 2: Polling. Agents periodically re-query the memory store ("Has anything changed?"). This creates unnecessary load when nothing has changed, adds latency when something has (the agent won't know until the next poll), and doesn't scale — N agents polling at T intervals generates N/T queries per second.

Approach 3: Broadcast all events to all agents. A naive pub/sub approach where every write event is sent to every connected agent. This causes information overload — a monitoring

agent doesn't need the personal agent's "user prefers dark mode" update, and a personal agent doesn't need the monitoring agent's "server CPU at 90%" alert. Each agent's operational scope is different, and unscoped broadcasts waste context window capacity and cognitive processing.

Approach 4: Agent-to-agent messaging. Agents explicitly send messages to specific other agents. This requires agents to know about each other, creates tight coupling, and doesn't handle the common case where the writing agent doesn't know (or care) who needs its information.

5.3 The Unsolved Problem

No existing system provides a method for:
1. Automatically routing memory change events only to agents whose operational scope overlaps with the change, AND
2. Preventing an agent from receiving notifications of its own writes (which would be redundant — it already knows what it wrote), AND
3. Doing this without polling, broadcasting, or explicit agent-to-agent addressing, AND
4. Allowing each agent to define its scope declaratively (by target, category, and key prefix), rather than imperatively (by writing routing code).

---

6. Summary of the Invention

The invention, called **Telepathy**, is a scoped event routing system for multi-agent shared memory. It provides real-time awareness between autonomous AI agents operating on a shared memory store, where:

- **Agents subscribe** to changes matching their operational scope (target, category, key prefix).
- **The daemon routes** change events only to agents whose subscriptions match the change.
- **Self-exclusion** prevents an agent from receiving its own write events.
- **Hot block invalidation** triggers re-computation of an agent's working context when a relevant change arrives from another agent.

The result is that agents gain shared awareness — "telepathy" — without polling, broadcasting, or explicit inter-agent communication. Agent B writes a memory; Agent A's working context updates within milliseconds, automatically.

---

7. Detailed Description

7.1 System Architecture

The system comprises three components:

1. **A shared memory daemon** (single process) that owns the memory store (an SQLite database with full-text search) and serves all agents via gRPC.
2. **A pub/sub hub** (in-process, within the daemon) that maintains subscriptions and routes events.
3. **Agent-side client libraries** that subscribe to change streams and maintain hot block caches.

All agents connect to the same daemon instance. The daemon is the sole writer to the database, which eliminates write contention and ensures serialization of all mutations.

7.2 Subscription Model

Each agent declares a **subscription** that specifies which memory changes it cares about, using three dimensions:

Dimension	Description	Example
Target	The memory namespace: "memory" (general knowledge) or "user" (personal user data)	A dev agent subscribes to "memory"; a personal agent subscribes to both
Category	The type of memory: "environment", "project", "quirk", "observation", "user"	An agent boosting "project" and "environment" categories in its hot block subscribes to those categories
Key Prefix	A prefix match on the key field for fine-grained scoping	An agent watching server configuration subscribes to key prefix "server-"

A subscription that omits a dimension matches all values for that dimension. This allows agents to declare broad or narrow scope:

- Broad: `{ target: "memory" }` — receives all changes to general knowledge.
- Narrow: `{ target: "memory", categories: ["project", "environment"], key_prefixes: ["server-"] }` — receives only project and environment changes whose keys start with "server-".

7.3 Self-Exclusion

When an agent writes a memory (e.g., `AddMemory`), the daemon publishes a change event to the pub/sub hub. The hub's routing logic explicitly checks whether the subscription's `agent_id` matches the event's `source_agent_id`. If they match, the event is withheld from that subscriber.

Why this matters: Without self-exclusion, every write by Agent A would cause Agent A to receive a notification about its own write. This is redundant (the agent already has the information in its context) and creates a feedback loop (Agent A writes → Agent A receives notification → Agent A may write again based on the notification → Agent A receives another notification → ...). Self-exclusion breaks this loop at the routing layer, not at the application layer — agents don't need to write filtering code.

7.4 Hot Block Invalidation

Each agent maintains a **hot block** — a compact, rendered text block of its most important memories, injected into the LLM's system prompt. The hot block is computed by the daemon based on per-agent parameters:

- Character limit (e.g., 4000 characters for memory, 2000 for user data)
- Minimum priority threshold (e.g., priority ≥ 2)
- Category boost list (e.g., boost "project" and "environment" for a dev agent)
- Category suppress list (e.g., suppress "observation" for a dev agent)

When a telepathy event arrives at an agent's subscription stream, the agent knows that a memory relevant to its hot block may have changed. The agent can then request a fresh hot

block from the daemon (via `GetHotBlock` RPC), which recomputes the block with the latest data.

Alternatively, the daemon can pre-compute the updated hot block and include it in the event itself (as a `HOT_BLOCK_CHANGED` event type), eliminating the round-trip. This is the preferred mode for low-latency operation.

7.5 Event Lifecycle

1. Agent A connects to daemon, calls `Subscribe` with its scope parameters.
2. Daemon opens a server-side streaming RPC to Agent A.
3. Agent B calls `AddMemory` to write "server IP is 10.0.0.5" with target="memory", category="environment", key="server-ip".
4. Daemon persists the memory to SQLite.
5. Daemon publishes an event `{type: ADDED, target: "memory", category: "environment", key: "server-ip", agent_id: "agent-b"}` to the pub/sub hub.
6. Hub evaluates all subscriptions:
   - Agent A's subscription `{target: "memory"}` matches. Agent A ≠ Agent B, so it's not excluded. Event is sent.
   - Agent B's own subscription (if any) matches on target, but is excluded because `agent_id == source_agent_id`.
7. Agent A receives the event on its subscription stream.
8. Agent A requests a fresh hot block from the daemon (or the event includes it).
9. Agent A replaces its cached hot block with the fresh one.
10. On Agent A's next LLM call, the system prompt includes the updated hot block — Agent A now knows about Agent B's discovery, within milliseconds of when Agent B wrote it.

7.6 Disconnection Cleanup

When an agent disconnects (gracefully or by crash), the daemon detects the stream cancellation via the gRPC context. The daemon's `Subscribe` handler has a `defer`-based cleanup that removes the subscription from the hub. This prevents the hub from accumulating stale subscriptions and attempting to send events to disconnected agents.

The hub also provides an `UnsubscribeAll(agentID)` method that removes all subscriptions for a given agent, used when an agent's heartbeat times out or its connection is lost.

---

8. Novel Aspects and Distinctions from Prior Art

8.1 Scoped Routing vs. Broadcast Pub/Sub

Existing pub/sub systems (e.g., MQTT, NATS, Kafka) allow topic-based routing, but topics are opaque strings that subscribers must know in advance. The invention uses **structured, semantic scope matching** (target × category × key prefix) that maps directly to the agent's hot block computation parameters. This means:
- An agent's subscription specification is derived directly from its hot block configuration — no separate routing configuration is needed.
- The scope dimensions correspond to the memory schema, not to arbitrary topic strings.
- An agent that changes its hot block configuration (e.g., a dev agent switches to monitoring mode and wants to suppress "project" and boost "environment") automatically changes its subscription scope.

8.2 Self-Exclusion at the Routing Layer

Existing pub/sub systems deliver all matching events to all subscribers, including events published by the subscriber itself. Some systems allow filtering on the consumer side (e.g., Kafka consumer groups), but this requires application logic to identify and discard self-published events.

The invention performs self-exclusion at the routing layer, before the event is ever placed on the subscriber's channel. This:
- Eliminates the feedback loop at its source.
- Reduces unnecessary network traffic and processing.
- Prevents subtle bugs where an agent reacts to its own write and creates an infinite loop.
- Requires no application-layer filtering code in the agent.

8.3 Hot Block Invalidation via Event Stream

Existing agent systems either freeze the context at session start or poll for updates. The invention uses the event stream as an invalidation signal: when a relevant change arrives, the hot block is known to be stale and must be recomputed. This gives:

- **Lower latency than polling:** Events arrive within milliseconds of the write, not at the next poll interval.
- **Lower cost than polling:** No queries are made when nothing has changed.
- **Freshness guarantee:** The hot block is re-computed only when it is known to be stale, not on a timer.

8.4 Single-Writer Architecture with gRPC Streaming

The daemon is the sole writer to the SQLite database. All agents read and write through the daemon via gRPC. This eliminates write contention (no SQLITE_BUSY errors), ensures all writes trigger proper event publication (no writes bypass the hub), and allows the daemon to compute hot blocks atomically without read-write races.

The use of gRPC server-streaming (a persistent, bidirectional connection) provides real-time delivery without the overhead of HTTP polling or WebSocket framing.

---

9. Potential Claims

The following are potential claim areas for evaluation by patent counsel:

1. **A method for routing memory change events in a multi-agent system, comprising:** maintaining a set of subscriptions each defined by an agent identifier, a target scope, a category scope, and a key prefix scope; receiving a write event from a first agent, the write event including a target, a category, a key, and a source agent identifier; evaluating each subscription against the write event by matching the target, category, and key against the subscription's corresponding scopes; withholding the event from any subscription whose agent identifier matches the source agent identifier (self-exclusion); and delivering the event to subscriptions that match and are not self-excluded.

2. **The method of claim 1, wherein** the subscriptions are declarative — an agent specifies its scope without knowledge of other agents, and the routing is performed by a central daemon without inter-agent communication.

3. **The method of claim 1, wherein** the target, category, and key prefix scopes correspond directly to the parameters used to compute the agent's hot block (a compact, rendered summary of the agent's most relevant memories), such that an agent's subscription is derived from its hot block configuration without additional routing logic.

4. **A method for maintaining contextual awareness between autonomous AI agents, comprising:** maintaining a shared memory store accessible to a plurality of agents; computing, for each agent, a hot block comprising a subset of memories selected by priority, target, and category parameters; publishing an event when a memory is written to the shared store; selectively routing the event to agents whose hot block parameters overlap with the written memory, excluding the writing agent; and invalidating the hot block of each receiving agent responsive to the event.

5. **The method of claim 4, wherein** hot block invalidation triggers re-computation of the hot block by a central daemon, and the recomputed hot block is delivered to the agent as part of the event or via a separate request.

6. **A system for scoped event routing in a multi-agent shared memory environment, comprising:** a shared memory daemon configured to persist memories and compute hot blocks; a publish/subscribe hub within the daemon, the hub maintaining subscriptions that specify target, category, and key prefix scopes; self-exclusion logic within the hub that withholds events from the agent that originated the write; and a gRPC server-streaming connection between the daemon and each agent for real-time event delivery.

7. **The method of claim 1, further comprising** automatically cleaning up subscriptions when an agent disconnects, using context cancellation from the streaming connection.

---

10. Prior Art Search Areas

For the patent attorney's evaluation:

Patent Class	Search Terms
G06N (AI)	"multi-agent memory sharing", "agent context awareness", "agent hot block"
G06F 16/00 (Information retrieval)	"scoped publish subscribe", "semantic event routing"
H04L 67/00 (Network protocols)	"self-exclusion publish subscribe", "agent identity routing"
G06F 9/54 (Inter-process communication)	"shared memory notification", "invalidation-based cache update"

Known related systems to distinguish from:
- **MQTT/NATS/Kafka** — topic-based pub/sub, no structured scope matching, no self-exclusion
- **Redis Pub/Sub** — channel-based, no semantic scope, no self-exclusion
- **LangChain/LangGraph** — agent orchestration, but agents don't share memory through scoped events
- **AutoGen (Microsoft)** — multi-agent chat, but agents communicate via explicit messages, not scoped memory events
- **CrewAI** — agent delegation, but no shared memory with scoped invalidation
- **OpenAI Assistants API** — single-agent context, no multi-agent memory sharing
- **MemGPT/Letta** — single-agent memory management, no multi-agent telepathy

---

11. Commercial Application

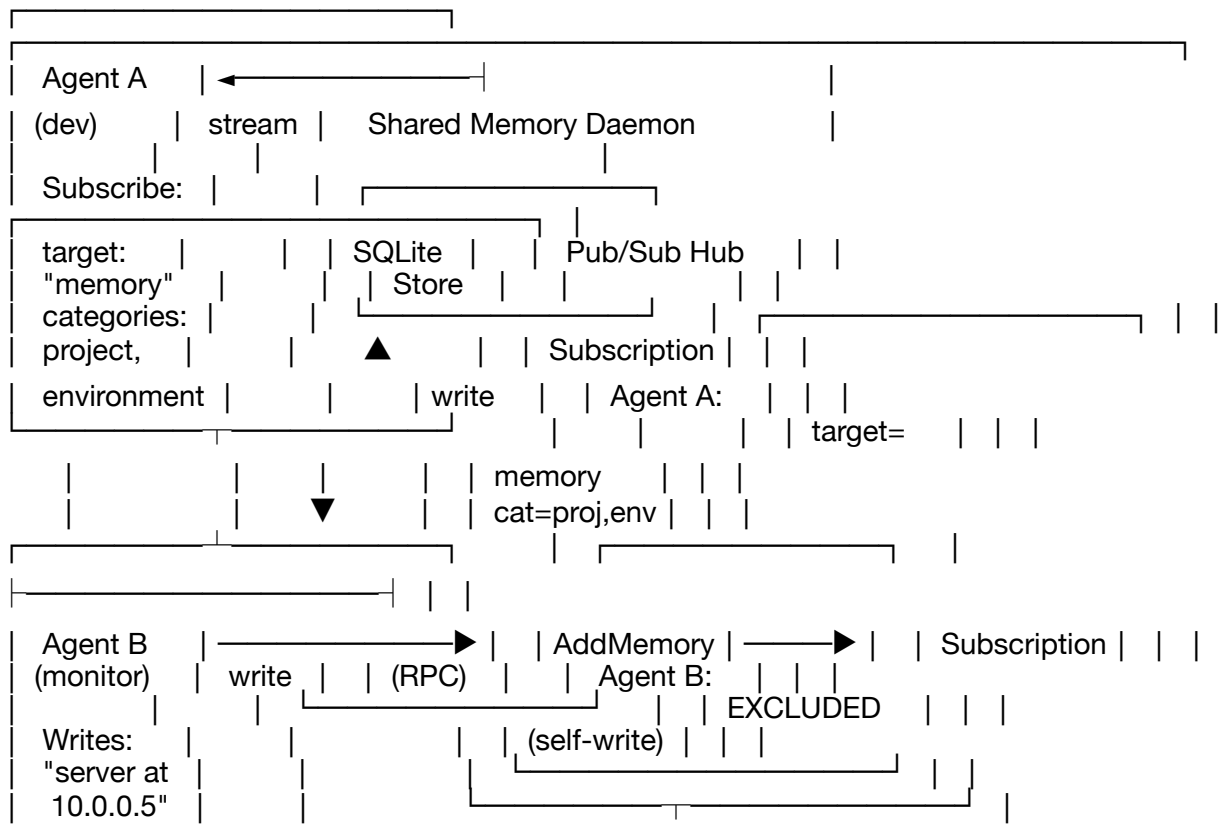
This invention is implemented in the **Mnemosyne** project (github.com/aj-nt/mnemosyne), an open-source shared memory daemon for AI agents. The telepathy system is the core differentiator that enables multi-agent awareness without polling, broadcasting, or explicit inter-agent messaging.

Potential commercial applications:
- Multi-agent coding teams (one agent writes architecture decisions, another agent reads them in real-time)
- DevOps automation (monitoring agent detects incident, development agent receives the alert in its working context)
- Personal AI assistants (one agent learns user preferences, all other agents reflect them immediately)
- Enterprise knowledge management (department-specific agents share relevant findings across team boundaries)

---

12. Drawings / Figures

Figure 1: System architecture showing the daemon, pub/sub hub, and multiple agents connected via gRPC streaming.



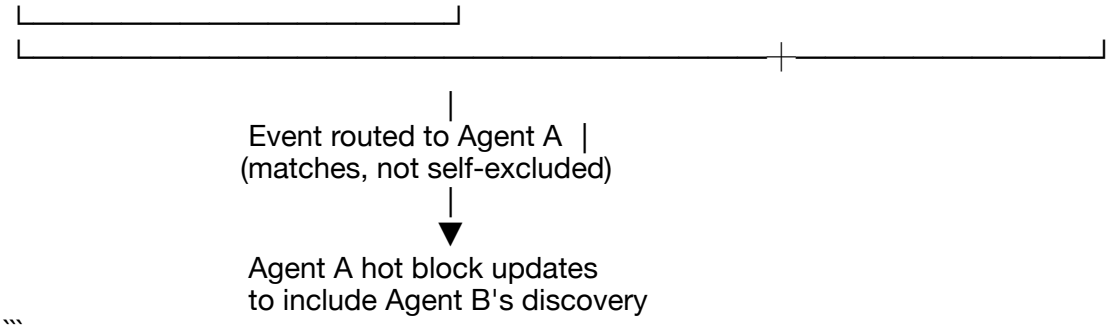


Figure 2: Self-exclusion routing logic.

```
Write Event: {target: "memory", category: "environment",
              key: "server-ip", source: "agent-b"}

Subscription Evaluation:
┌──────────────────────────────────────────────────────────────────┐
│ Agent A subscription:                                            │
│   agent_id: "agent-a"                                            │
│   target: "memory"                                               │
│   category: "project", "environment"                             │
│ ──────────────────────────────────────────────────────────────── │
│ Target match?    target="memory" ✓                               │
│ Category match?  "environment" ✓                                 │
│ Self-exclusion? "agent-a" ≠ "agent-b" ✓                          │
│ ──────────────────────────────────────────────────────────────── │
│ ROUTE → Agent A receives event                                   │
└──────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────┐
│ Agent B subscription (if any):                                   │
│   agent_id: "agent-b"                                            │
│   target: "memory"                                               │
│ ──────────────────────────────────────────────────────────────── │
│ Target match?    target="memory" ✓                               │
│ Category match?  (none = all) ✓                                  │
│ Self-exclusion? "agent-b" = "agent-b" ✗                          │
│ ──────────────────────────────────────────────────────────────── │
│ WITHHOLD — Agent B does NOT receive                              │
│ its own write event                                              │
└──────────────────────────────────────────────────────────────────┘
```

Figure 3: Hot block invalidation lifecycle.

```
```

Agent A's hot block contains:
  [environment] server-ip: 10.0.0.1
  [project] mnemosyne: Memory daemon in Go

Agent B writes: "server-ip is now 10.0.0.5"

Event arrives at Agent A → hot block known stale

Agent A requests fresh hot block from daemon

Agent A's updated hot block:
  [environment] server-ip: 10.0.0.5   ← UPDATED
  [project] mnemosyne: Memory daemon in Go

Agent A's next LLM call includes the updated context
→ Agent A now knows about the server change
→ Without polling, without being told, without asking
→ Telepathy.
```

---

## 13. Implementation Reference

The invention is implemented in Go, approximately 160 lines for the pub/sub hub, 50 lines for the subscription routing logic, and 30 lines for the self-exclusion check. The daemon communicates with agents via gRPC server-streaming (protoc-generated code). The full source code is available at github.com/aj-nt/mnemosyne under the MIT license.

Key source files:
- `daemon/internal/pubsub/hub.go` — Hub, Subscribe, Publish, matches, UnsubscribeAll
- `daemon/internal/grpc/service.go` — Subscribe RPC, AddMemory (with event publishing)
- `client/hotblock_cache.go` — Client-side hot block cache with Update method
- `proto/mnemosyne.proto` — SubscribeRequest, UpdateEvent, RegisterAgentRequest messages

---

*Prepared by the inventor for evaluation by patent counsel. This document is a technical disclosure, not a patent application.*
```